Code walkthrough

tb_control_unit.v

Testbench: Control Unit

What it does	Checks the decoder and branch unit (22 checks)
Where it works	module-level test
Size	117 lines, explained in 9 small parts
Modules in this file	tb_control_unit
Calls (uses)	control_unit.v (module control_unit)
Called by	nobody - this is a testbench (top of simulation)

This testbench checks every row of the decoder truth table (all 16 opcodes) and every case of the branch unit. The control unit is combinational, so no clock is needed.

Map of the file

Part	Lines	What this part is
1	1-12	Header
2	13-26	Signals
3	28-35	The design under test
4	37-40	Packing the outputs
5	42-59	Task: check_op
6	61-77	Task: check_branch
7	79-100	Part A: decoder truth table
8	102-109	Part B: branch unit
9	111-117	Summary

> How to read this document: each part below shows a few lines of the real file, with their line numbers, followed by a plain-language explanation. Read the parts in order to understand the whole file without scrolling through it in one go.

Part 1 · Header (lines 1-12)

tb_control_unit.v lines 1-12

```
 1  `timescale 1ns / 1ps
 2  //
 3  // CONNECTIONS
 4  //   Calls       : control_unit.v (module control_unit)
 5  //   Called by   : nobody - this is a testbench (top of simulation)
 6  // ========================================================================
 7  // FILE: tb_control_unit.v
 8  // MODULE-LEVEL TEST 3: the control unit (no clock needed).
 9  // Part A: decoder output for EVERY opcode, compared with the
10  //         control-signal truth table in the report.
11  // Part B: branch unit - when is a jump taken and the pipeline flushed?
12  // ========================================================================
```

- Time unit and description: Part A tests the decoder, Part B the branch unit.
- The **CONNECTIONS** lines at the end of the header list which other files this file uses (Calls) and which files use it (Called by).

Part 2 · Signals (lines 13-26)

tb_control_unit.v lines 13-26

```
13  module tb_control_unit;
14
15      reg  [3:0] opcode    = 4'b0000;
16      reg        ex_is_jmp = 0;
17      reg        ex_is_jz  = 0;
18      reg        zero_flag = 0;
19
20      wire [2:0] alu_ctrl;
21      wire       alu_src_sel, wb_sel, reg_we, mem_we;
22      wire       zero_we, carry_we, is_jmp, is_jz;
23      wire       pc_sel, flush;
24
25      integer errors = 0;
26      integer tests  = 0;
```

Inputs for both parts (opcode for the decoder; ex_is_jmp, ex_is_jz, zero_flag for the branch unit) and wires for all outputs.

Part 3 · The design under test (lines 28-35)

tb_control_unit.v lines 28-35

```
28      // CALLS: module control_unit from file control_unit.v
29      control_unit DUT (
30          .opcode(opcode), .alu_ctrl(alu_ctrl), .alu_src_sel(alu_src_sel),
31          .wb_sel(wb_sel), .reg_we(reg_we), .mem_we(mem_we),
32          .zero_we(zero_we), .carry_we(carry_we), .is_jmp(is_jmp), .is_jz(is_jz),
33          .ex_is_jmp(ex_is_jmp), .ex_is_jz(ex_is_jz), .zero_flag(zero_flag),
34          .pc_sel(pc_sel), .flush(flush)
35      );
```

One control unit with every port connected.

→ This part **calls module** `control_unit` from file `control_unit.v`. How that module works is explained in `Explained_control_unit.pdf`.

Part 4 · Packing the outputs (lines 37-40)

tb_control_unit.v lines 37-40

```
37      // All decoder outputs packed into one 11-bit word, in this order:
38      // alu_ctrl[2:0] alu_src_sel wb_sel reg_we mem_we zero_we carry_we is_jmp is_jz
39      wire [10:0] signals = {alu_ctrl, alu_src_sel, wb_sel, reg_we, mem_we,
40                             zero_we, carry_we, is_jmp, is_jz};
```

The nine decoder outputs are joined into one 11-bit word with { } (concatenation). Then one comparison checks all of them at once.

Part 5 · Task: check_op (lines 42-59)

tb_control_unit.v lines 42-59

```
42      // check the decoder output for one opcode
43      task check_op;
44          input [8*6:1] name;
45          input [3:0]   op;
46          input [10:0]  expected;
47          begin
48              opcode = op;  #1;
49              tests = tests + 1;
50              if (signals == expected)
51                  $display("PASS  %0s alu=%0d src=%b wb=%b rwe=%b mwe=%b zwe=%b cwe=%b jmp=%b jz=%b",
52                           name, alu_ctrl, alu_src_sel, wb_sel, reg_we, mem_we,
53                           zero_we, carry_we, is_jmp, is_jz);
54              else begin
55                  $display("FAIL  %0s got %b, expected %b", name, signals, expected);
56                  errors = errors + 1;
57              end
58          end
59      endtask
```

Sets the opcode, waits 1 ns, compares the packed word with the expected one and prints the signals.

Part 6 · Task: check_branch (lines 61-77)

tb_control_unit.v lines 61-77

```
61      // check the branch unit for one situation
62      task check_branch;
63          input [8*24:1] name;
64          input          jmp, jz, z;
65          input          exp_pc_sel;
66          begin
67              ex_is_jmp = jmp;  ex_is_jz = jz;  zero_flag = z;  #1;
68              tests = tests + 1;
69              if (pc_sel == exp_pc_sel && flush == exp_pc_sel)
70                  $display("PASS  %0s pc_sel=%b flush=%b", name, pc_sel, flush);
71              else begin
72                  $display("FAIL  %0s pc_sel=%b flush=%b (expected %b)",
73                           name, pc_sel, flush, exp_pc_sel);
74                  errors = errors + 1;
75              end
76          end
77      endtask
```

Sets is_jmp, is_jz and Zero, waits 1 ns, and checks that pc_sel and flush both equal the expected value.

Part 7 · Part A: decoder truth table (lines 79-100)

tb_control_unit.v lines 79-100

```
 79      initial begin
 80          // The tasks 'check_op' and 'check_branch' used below are defined
 81          // above in this same file.
 82          $display("\n===== tb_control_unit: control unit test =====");
 83          $display("--- Part A: decoder (DECODE stage) ---");
 84          //          name     opcode    alu   src   wb   rwe  mwe  zwe  cwe  jmp  jz
 85          check_op("ADD   ", 4'b0000, {3'd0, 1'b0, 1'b0, 1'b1, 1'b0, 1'b1, 1'b1, 1'b0, 1'b0});
 86          check_op("SUB   ", 4'b0001, {3'd1, 1'b0, 1'b0, 1'b1, 1'b0, 1'b1, 1'b1, 1'b0, 1'b0});
 87          check_op("AND   ", 4'b0010, {3'd2, 1'b0, 1'b0, 1'b1, 1'b0, 1'b1, 1'b0, 1'b0, 1'b0});
 88          check_op("OR    ", 4'b0011, {3'd3, 1'b0, 1'b0, 1'b1, 1'b0, 1'b1, 1'b0, 1'b0, 1'b0});
 89          check_op("XOR   ", 4'b0100, {3'd4, 1'b0, 1'b0, 1'b1, 1'b0, 1'b1, 1'b0, 1'b0, 1'b0});
 90          check_op("NOT   ", 4'b0101, {3'd5, 1'b0, 1'b0, 1'b1, 1'b0, 1'b1, 1'b0, 1'b0, 1'b0});
 91          check_op("LDI   ", 4'b0110, {3'd6, 1'b1, 1'b0, 1'b1, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0});
 92          check_op("MOV   ", 4'b0111, {3'd6, 1'b0, 1'b0, 1'b1, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0});
 93          check_op("LOAD  ", 4'b1000, {3'd6, 1'b1, 1'b1, 1'b1, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0});
 94          check_op("STORE ", 4'b1001, {3'd6, 1'b1, 1'b0, 1'b0, 1'b1, 1'b0, 1'b0, 1'b0, 1'b0});
 95          check_op("JMP   ", 4'b1010, {3'd0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b1, 1'b0});
 96          check_op("JZ    ", 4'b1011, {3'd0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b1});
 97          check_op("RES-C ", 4'b1100, {3'd0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0});
 98          check_op("RES-D ", 4'b1101, {3'd0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0});
 99          check_op("RES-E ", 4'b1110, {3'd0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0});
100          check_op("RES-F ", 4'b1111, {3'd0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0});
```

One line per opcode, written in the same order as the truth table in the report: ALU code, alu_src_sel, wb_sel, reg_we, mem_we, zero_we, carry_we, is_jmp, is_jz. The four reserved opcodes must give all zeros.

Part 8 · Part B: branch unit (lines 102-109)

tb_control_unit.v lines 102-109

```
102          $display("--- Part B: branch unit (EXECUTE stage) ---");
103          //              name                      jmp jz  Z   pc_sel
104          check_branch("no jump instruction     ", 0, 0, 0, 0);
105          check_branch("no jump, Zero = 1       ", 0, 0, 1, 0);
106          check_branch("JMP                     ", 1, 0, 0, 1);
107          check_branch("JMP (Zero = 1)          ", 1, 0, 1, 1);
108          check_branch("JZ with Zero = 0        ", 0, 1, 0, 0);
109          check_branch("JZ with Zero = 1        ", 0, 1, 1, 1);
```

The six situations: no jump (Zero 0 or 1), JMP (Zero 0 or 1), JZ with Zero = 0 (no jump) and JZ with Zero = 1 (jump and flush).

Part 9 · Summary (lines 111-117)

tb_control_unit.v lines 111-117

```
111          if (errors == 0)
112              $display("tb_control_unit PASSED: %0d of %0d checks OK\n", tests, tests);
113          else
114              $display("tb_control_unit FAILED: %0d of %0d checks failed\n", errors, tests);
115          $finish;
116      end
117  endmodule
```

Prints PASSED / FAILED and ends the simulation.

> **In short:** 16 decoder checks + 6 branch checks, all passing.